RFC: Shared Chunk Cache Dispatch Module

alex.bryan@lifeboat.llc
clay.carper@lifeboat.llc
john.mainzer@lifeboat.llc

The objective of the multi-thread safe shared chunk cache is to maximize I/O throughput. However, to avoid corruption we must serialize concurrent writes to the same location. While concurrent reads from a given location should be allowed, for similar reasons, we must prevent concurrent reads and writes to a given location.

Further, the memory allocated to the shared chunk cache is finite. To avoid thrashing and the resulting performance degradation, we must ensure that the I/O threads active in the shared chunk cache at any point in time will not overtax this resource.

To address these issues, this RFC proposes queuing I/O requests on receipt, determining their targets and shared chunk cache memory requirements, and then dispatching them to the shared chunk cache when it is safe to do so.

Table of Contents

1 Introduction

The current implementation of the shared chunk cache is serial only, but it was designed with the future potential for multi-thread operations in mind. While the details of this upgrade are discussed in the shared chunk cache RFC, there are some issues that are best handled slightly above the level of the shared chunk cache proper.

This RFC is directed at two of these:

- Ensuring that writes to the same location are serialized, and that any (possibly concurrent) reads to a given location are executed either before or after writes to same. While it is not addressed in this version of the RFC, there is also the matter of disabling I/O to datasets while they are being resized.

- Metering dispatch of work to the shared chunk cache to avoid thrashing

Both of these issues can be addressed by queuing incoming I/O requests just before they reach the shared chunk cache, and then dispatching them when it is safe to do so.

While it is an obvious extension, this version of the RFC only considers safe dispatch of I/O requests in receipt order. While re-ordering I/O requests to maximize throughput while maintaining correctness is an obvious optimization, it is not contemplated in this version of the RFC.

2 Conceptual Overview

Conceptually, the dispatch module has two operations – I/O request ingestion, and I/O request dispatch.

Upon entry of a thread with an I/O request, the dispatch module must:

- Look up and store the necessary data; the shared chunk cache space requirements, type, and target of the request.

- Insert the I/O request with the necessary data at the tail of the dispatch queue.

Following queue insertion, the thread must wait until either the I/O request is dispatched, or the I/O request completes, depending on if the shared chunk cache uses a thread pool to execute I/O requests.

For now, I/O requests will be dispatched only from the head of the I/O request queue. The dispatch code is executed on an entry in the dispatch queue if either:

- The entry has just moved to the head of the dispatch queue, or

- The entry is already at the head of the queue and a previously dispatched I/O request has just completed.

Once triggered, the dispatch code will dispatch the I/O request at the head of the dispatch queue if the following conditions are met:

1. Either the shared chunk cache is quiescent, or the sum of the shared chunk cache memory requirements of the active I/O requests and that of the candidate are less than the maximum active memory allocation for the shared chunk cache.

2. Either no active I/O request targets the dataset(s) targeted by the candidate request, or the candidate I/O request is a read, and all active requests that share target datasets with the candidate are reads as well.

If these conditions are met, the I/O request at the head of the dispatch queue is moved to the active I/O request list and dispatched[1]. Note that this triggers examination of the next entry on the dispatch queue.

When an I/O request completes[2], it must be removed from the active I/O request list before the dispatch code is triggered. This is necessary, as the active I/O request list will serve as ground truth for the target dataset(s) and shared chunk cache memory requirements of all active I/O requests.

Condition 2 (above) is more restrictive than necessary. As long as any active read or write I/O requests don't touch the portions of the dataset targeted by the candidate, the candidate can be dispatched safely. However, this complicates the dispatch code, and it is not clear that this optimization will be worth the effort in most cases. Thus we leave this potential optimization for later consideration.

With this overview in hand, we proceed to more detailed discussions of the ingestion and dispatch operations.

2.1 Ingestion

Whether the I/O request is a read or a write, the target dataset(s) are trivial to determine. However, estimating the shared chunk cache space requirements is a more involved process. This is particularly true for structured chunks containing sparse data—the initial target of the shared chunk cache. The goal of ingesting this data for estimation is to ensure that I/O requests are processed in receipt order with the opportunity to reorder some requests in the future if it can improve dispatch throughput to the Shared Chunk Cache [4] (SCC). Dispatching requests through the SCC (performed through the H5SC_read and H5SC_write functions) allows the use of a LRU cache while performing I/O with aims of improving performance and multi-thread security.

2.1.1 Read Requests

This is the easiest case. Here we first create a list of each chunk in each dataset addressed by the read request. With this done, we must look up each chunk on this list in its host dataset chunk index, obtain the uncompressed size of its image on disk in bytes, and compute the sum over all such chunks. This

[1] The exact details of dispatching an I/O request are TBD pending implementation of the multi-thread version of the shared chunk cache.

[2] As with dispatch, the details of I/O request completion are TBD. Hence we only discuss those elements necessary for the dispatch module.

value, possibly times an empirically derived fudge factor, is the estimate shared chunk cache space requirements for the read request.

Read request dispatch pseudo-code below.

```
start function H5SC_dispatch_read()
  for each dataset
    call₁ H5SC__io_info_init()
    multiply selected chunks count (from call₁) and chunk size
    add above value to total estimation
  end for
  call LFDLL enqueue operation
  while enqueued request is not the head of the queue
    wait for condition variable
  end while
  obtain lock
  if a dataset touched by the request has been modified since
     estimation
    recalculate estimation
  end if
  call H5SC_read()
  call LFDLL dequeue operation
  release lock
end function H5SC_dispatch_read()
```

The above pseudo-code performs a pessimistic estimation (by assuming chunks are dense) on the cache space necessary to perform a read operation through the Shared Chunk Cache. This dense assumption is preferred because assuming that chunks are sparse may lead to dense chunks blowing up the cache. Following this estimation, a fudge-factor may be applied in future versions. The size estimation of a request currently has no impact on queue ordering, but any future logic used for reordering certain requests will need these estimations. Subportioning of large requests is done by H5SC_read(), and therefore potential thrashing caused by dispatching a request to the Shared Chunk Cache does not need considered here and is instead handled by the SCC.

2.1.2 Write Request

Computation of shared chunk cache space requirements for write requests is complicated by the fact that for sparse data or variable length data stored in structured chunks, the write may change the size of the chunk. This may trigger thrashing in the shared chunk cache while it tries to resize its chunks and stay within its space allocation simultaneously.

Deriving a tight limit on this size increase, while likely doable, will be involved and may or may not be worth the effort. Thus, for now, we will content ourselves with computing an upper bound on the maximum possible shared chunk cache space requirements by computing the sum of the current sizes of the target chunks as per read requests (above), and adding to it the total number of bytes written. This will be a tight bound for initial writes to the target dataset(s) and a somewhat pessimistic bound for writes which include modifications to existing data.

Determining the number of bytes in a given write request requires the following: determining the number of elements in the selections for each target dataset, determining the size of elements in the target datasets, performing the necessary multiplication of these values, and summing across the target datasets if necessary. This value, added to the sum of the current target chunk sizes and possibly multiplied by an empirically derived fudge factor, gives us our estimated shared chunk cache space requirement for the write request.

Write request dispatch pseudo-code below.

```
start function H5SC_dispatch_write()
  for each dataset
    add the dataset's buffer size to the total incoming data
        estimation
    call₁ H5SC__io_info_init()
    call₂ dataset layout_query callback
    multiply selected chunks count (from call₁) and chunk size
    add above value to total estimation
  end for
  add total space estimation and total incoming data estimations
  call LFDLL enqueue operation
  while enqueued request is not the head of the queue
    wait for condition variable
  end while
  obtain lock
  if a dataset touched by the request has been modified since
     estimation
    recalculate estimation
  end if
  call H5SC_write()
  call LFDLL dequeue operation
  release lock
end function H5SC_dispatch_write()
```

Similar to read requests, A pessimistic estimation is being made. By including the full chunk size and the size of all buffered data in this estimation, the assumption being made is that chunks are dense and writing more data necessitates additional space to be made. In this case, the inclusion of the buffer size would represent the data contained by these new chunks, so the risk of thrashing is mitigated. No fudge-factor is being included currently, but that may change. Currently, the estimation being made is not impacting the ordering of requests moving through the queue, since subportioning of large requests is being performed by H5SC_write(). In the future, this estimation in tandem with calculations on the available cache space may inform the reordering of some requests to improve throughput, if any such improvement exists.

2.1.3 Potential Interactions

It is worth noting that I/O requests on the dispatch queue but not yet executed may change the size estimations discussed above. As a result, one can argue that we should delay computing these estimates until just before dispatch. If it turns out that re-ordering request to maximize throughput is not worth the effort, this will be the way to go. Since it is likely that the majority of I/O request will require all available shared chunk cache memory, it will be no great surprise if this turns out to be the case.

That being said, it is best to keep our options open if it can be done cheaply. To do this, we will require code to be executed at the completion of write requests to scan the dispatch queue and mark any I/O request whose shared chunk cache space requirements may have changed.

Observe also that the census of chunks touched by the I/O request created as part of the shared chunk cache space requirements estimation duplicates elements of the processing performed by the shared chunk cache when it receives a new I/O request. While we should avoid premature optimization, thought should be given to allowing the shared chunk cache to reuse the results of the space requirements calculation to avoid redundant computations.

2.2 Dispatch

Since the multi-thread version of the shared chunk cache doesn't exist yet, we can't really discuss the details of the dispatch operation. We can, however, list the required operations and tests to be performed before an I/O request is dispatched.

Recall that the dispatch code must be triggered whenever an I/O request moves to the head of the dispatch queue, or immediately after an active I/O request completes.

1. Check to see if the I/O request is marked to indicate that its shared chunk cache memory requirements may have changed. Re-compute if so, and reset the mark.
2. If the active I/O requests list is empty, dispatch the candidate, and restart the dispatch routine with the new head of the dispatch queue. Otherwise:
3. Scan the active I/O requests list to determine the total shared chunk cache space requirements of the active I/O requests. Subtract this value from the maximum space allocated to the shared chunk cache less any space reserved open datasets not being read or written at present. If this value is less than the shared chunk cache space requirements of the candidate I/O request, do nothing and sleep pending the next dispatch trigger.

4. Scan the active I/O request list to determine if any of the active I/O requests target any of the datasets targeted by the candidate I/O request. If none do, or if all such active I/O requests are reads and the candidate is a read as well, dispatch the candidate. Otherwise, do nothing and sleep pending the next trigger.

Observe that in case 2 we will cheerfully dispatch an I/O request with shared chunk cache space requirements larger than the available space in the shared chunk cache. This is is a common occurrence in the serial version of the shared chunk cache with code to manage it. By allocating the entire shared chunk cache to large I/O requests, we can use the existing serial space management code without change. Note that we may want to revisit this, as it is possible that we will get better though put by modifying the shared chunk cache to handle the oversubscribed use case without thrashing. However, for now it seems prudent to simply avoid the issue until we have a working multi-thread shared chunk cache and the time and resources to experiment.

2.2.1 Queuing Requests

Following the estimation behavior discussed above, dispatching a request to be passed through the Shared Chunk Cache is agnostic to the type of request being made. Following the initial size estimation, the entire request (regardless of whether it will cause thrashing or not) is enqueued to a lock-free doubly-linked list (LFDLL). After the request is placed on the queue, the process waits for the queue's condition variable to signal that the request may have reached the front of the queue. When it does, the process whose element is now at the front of the queue obtains the lock and resumes, recalculating its size estimation if necessary and passing the request through the Shared Chunk Cache. Once the request is done, the request will be dequeued and the lock released, either clearing the queue if it was the last element or allowing another processes' request to become the new queue head for processing.

As a note, the queue and lock being discussed in this section are initialized and destroyed alongside the cache that owns it. If multiple files are open at once (meaning there are multiple caches) requests will be enqueued to whichever queue they belong to.

3 Implementation Details

4 Testing

4.1 Current Unit Tests

Testing is performed to ensure that multi-thread simulated SWMR I/O requests are placed in a Lock-Free Queue in receipt order, and requests in the queue are being removed from the queue to be executed by the process that created the request. After a process adds a request to the queue, it sleeps until notified that the front of the queue has changed. If the new front of the queue is the request it is waiting to perform, the request is removed from the front of the queue and operation is continued. Removing a request from the front of the shared queue signals sleeping threads that the front of the queue has changed, then the request is executed if possible. These requests are randomly generated

using an optional user-supplied seed for reproducability, and the number of requests, number of threads, and verbosity of the execution are all optional arguments to the execution of this test.

4.2 Needed Unit Tests

Other

4.3 Needed Integration Tests

Other

Acknowledgment

This work is supported by the U.S. Department of Energy, Office of Science under Award number DE-SC0023583 for SBIR project “Supporting Sparse Data in HDF5”.

References

1) Carper, C. and Mainzer, J. “RFC: Shared Chunk Cache Design,” https://github.com/LifeboatLLC/SparseHDF5/blob/main/design_docs/08_28_2025_Shared_Chunk_Cache_RFC.pdf (August 28, 2025).
2) Mainzer, J. and Pourmal, E. “RFC: New Requirements for HDF5 Chunk Cache,” https://github.com/LifeboatLLC/SparseHDF5/blob/main/design_docs/RFC-HDF5-Chunk-Cache-Req-2023-12-18.pdf (December 18, 2023).
3) Fortner. N., Mainzer. J., Choi, V., and Carper, C. “RFC: Shared Chunk Cache Internal API,” https://github.com/LifeboatLLC/SparseHDF5/blob/main/design_docs/Shared_Chunk_Cache_API.v11.pdf (June 23, 2025).

Revision History

October 27, 2025:	Version 0.1 circulated for comment within Lifeboat, LLC.
November 21, 2025	Version 0.2 circulated for comment within Lifeboat, LLC.
November 24, 2025	Version 0.3 circulated for comment within Lifeboat, LLC.
January 23, 2025	Version 0.4 circulated for comment within Lifeboat, LLC.
February 18, 2025	Commentary added on SCC operations with links to docs.